

Edge cases (proving gather actually waits) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what a real group of unequal-length calls costs you). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need two async tasks — one quick, one slow — run together with `asyncio.gather`, and proof of when each one actually finished.

Here's the part that trips everyone up the first time: it's easy to assume `gather` hands back results as soon as the *first* task is done. It doesn't. `gather` waits for **every** task you gave it before it returns anything at all — even if one finished almost instantly and the rest are still running.

Things to use:

- `await asyncio.sleep(0.1)` — a short, fast task.
- `await asyncio.sleep(5)` — a long, slow task.
- `time.perf_counter()` — a timestamp you print at the moment each thing finishes.
- `await asyncio.gather(fast_task(), slow_task())` — run both, wait for both.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about one behavior of `asyncio.gather` : it runs its arguments concurrently, but it does not *return* until all of them have completed — its overall wall-clock time is bounded by whichever task takes longest, not whichever finishes first.

Both `fast_task` and `slow_task` use `await asyncio.sleep(...)` to simulate work, and both print a timestamp the moment they finish, before returning their value. The `main` coroutine records a start time, calls `await asyncio.gather(fast_task(), slow_task())` , then records and prints the time immediately after `gather` returns.

The exact pieces:

- `async def fast_task() -> str:` — `await asyncio.sleep(0.1)` , then print a timestamp, then return.
- `async def slow_task() -> str:` — same shape, `await asyncio.sleep(5)` .
- `time.perf_counter()` — printed at each finish point, and once right after `gather` returns.
- `await asyncio.gather(fast_task(), slow_task())` — the `await` here is what makes `main` pause until `gather` itself is done, which is exactly the 5 seconds set by `slow_task` , not the 0.1 seconds set by `fast_task` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what this means once "gather waits for the slowest" isn't 2 tasks but a group of 5 or 10 independent calls, and one of them is unusually slow — a flaky network path, an overloaded endpoint, whatever. Every fast task in that group finished its work long ago and is just sitting there, done, while the whole group's result stays locked behind the one slow straggler. The group's cost isn't its average task — it's its *worst* task, every single time.

The real design question isn't just "does gather wait for the slowest" (yes, always) — it's "should a task genuinely be allowed to take an unbounded amount of time inside a group I'm waiting on, or should something cap how long any one member of the group can hold up the rest?"

The extra piece that answers that question:

- **A per-task timeout, applied before the task ever reaches `gather`** : wrap each individual coroutine in `asyncio.wait_for(coro, timeout=...)` *before* passing it to `gather` — not around the whole `gather` call. That way, one genuinely hung task times out and fails on its own, instead of holding the entire group hostage indefinitely (`asyncio.gather(..., return_exceptions=True)` from the `gather_speed` exercise is what stops that one timeout from losing the other results too).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both prove `gather` waits for the slowest task, with a slow task that's still bounded (5 seconds, on purpose, so the exercise finishes). Advanced asks what happens if the "slow" task in a real group isn't bounded at all — and shows that the fix isn't wrapping `gather` itself in a timeout (that would cancel the fast tasks' already-finished results too), it's wrapping each individual task before it ever joins the group.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an async function called fast_task that takes nothing:
    wait 0.1 seconds (without blocking anything else)
    print the current time
    give back the value "fast"

make an async function called slow_task that takes nothing:
    wait 5 seconds (without blocking anything else)
    print the current time
    give back the value "slow"

make an async function called main that takes nothing:
    print the current time -> this is the start
    run fast_task and slow_task together with gather, and wait for both
    print the current time -> this is right after gather finished
    print whatever gather gave back

run main with asyncio.run(...)

look at the output:
    fast_task's print line appears almost immediately
    but the line right after gather doesn't appear until ~5 seconds later
```

Here's almost the whole thing — just try running it and reading it line by line:

```
import asyncio
import time

async def fast_task():
    await asyncio.sleep(0.1)
    print("fast_task finished at", time.perf_counter())
    return "fast"

async def slow_task():
    await asyncio.sleep(5)
    print("slow_task finished at", time.perf_counter())
    return "slow"
```

```
async def main():
    start = time.perf_counter()
    print("starting at", start)
    results = await asyncio.gather(fast_task(), slow_task())
```

Expected output if you run just this: nothing after the "starting at ..." line yet — add the print of the timestamp right after `gather` returns, the print of `results`, and `asyncio.run(main())` at the bottom.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import asyncio
import time

define:
    async def fast_task() -> str:
        await asyncio.sleep(0.1)
        print(f"fast_task finished at {time.perf_counter():.2f}")
        return "fast"

define:
    async def slow_task() -> str:
        await asyncio.sleep(5)
        print(f"slow_task finished at {time.perf_counter():.2f}")
        return "slow"

define:
    async def main() -> None:
        start = time.perf_counter()
        print(f"starting at {start:.2f}")
        results = await asyncio.gather(fast_task(), slow_task())
        print(f"gather returned at {time.perf_counter():.2f}")
        print(results)

if __name__ == "__main__":
    asyncio.run(main())
```

Fill in the missing pieces yourself, then compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
make a third task, truly_slow_task, that sleeps for 60 seconds (standing in for "
wrap it individually, before gather ever sees it:
    bounded_slow = asyncio.wait_for(truly_slow_task(), timeout=2)

run gather(fast_task(), bounded_slow, return_exceptions=True):
    fast_task succeeds
    bounded_slow's slot holds a TimeoutError, not a hang
    -> the whole group finishes in ~2 seconds, not 60
```

```
async def truly_slow_task() -> str:
    await asyncio.sleep(60)
    return "finally done"

async def main_bounded() -> None:
    bounded = asyncio.wait_for(truly_slow_task(), timeout=2)
    results = await asyncio.gather(fast_task(), bounded, return_exceptions=True)
    for result in results:
        if isinstance(result, Exception):
            print("a task timed out or failed:", result)
        else:
            print("a task succeeded:", result)
```

Time the whole `main_bounded()` call yourself to confirm it finishes in about 2 seconds, not 60, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's `slow_task` is slow but always bounded at 5 seconds — a stand-in that always finishes. Advanced adds a task that's unbounded on purpose (`truly_slow_task` , sleeping 60 seconds) and shows the fix isn't avoiding `gather` , it's wrapping the individual risky task in its own `asyncio.wait_for(...)` before it ever joins the group — so one runaway task costs you 2 seconds, not 60.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)